

Problème d'allocation de registre

Le fonctionnement de nos ordinateurs est un domaine attirant ma curiosité. Dans ma recherche de TIPE, je me suis donc tourné vers ce sujet, ce qui m'a amené au fonctionnement des langages de programmation et à la compilation. Là, j'y ai découvert le problème d'allocation de registre, qui m'a conquis.

Le problème d'allocation de registre s'intéresse à la façon de diminuer le nombre de cycle processeur nécessaire pour accéder à une variable. Plus le nombre de cycles nécessaire est important, moins efficace sera notre programme. En cela, il prend sa place dans le thème "Cycles, boucles".

Le candidat atteste avoir travaillé en monôme.

Positionnement thématique (ÉTAPE 1) :

- INFORMATIQUE (*Technologies informatiques*)
- INFORMATIQUE (*Informatique pratique*)
- INFORMATIQUE (*Informatique Théorique*)

Mots-clés (ÉTAPE 1) :

Mots-clés (en français) Mots-clés (en anglais)

<i>Registre</i>	<i>Register</i>
<i>Compilation</i>	<i>Compilation</i>
<i>Graphe d'interférence</i>	<i>Interference graph</i>
<i>k-coloration</i>	<i>k-coloration</i>
<i>Optimisation</i>	<i>Optimization</i>

Bibliographie commentée

Les **compilateurs** sont des outils essentiels dans le fonctionnement des langages de programmation. Ce sont les traducteurs qui permettent à nos machines de comprendre ce qu'on leur demande de faire. Ils possèdent trois phases : le *front-end*, le *middle-end* et le *back-end*.

Le *front-end* est propre à chaque langage. C'est la partie où on va lire et analyser le code, afin de le transformer en une représentation intermédiaire. On y trouve, dans cet ordre :

- L'analyse lexical, qui consiste à lire une première fois le code et à y placer des points de repère appelé *token* (lexem) pour identifier ce que sont chaque chose : s'agit-il d'une variable, d'un outil interne au langage ?
- Le *parsing* (analyse syntaxique) qui consiste à créer l'arbre de syntaxe à partir des *tokens* préalablement posés : on applique la grammaire du langage.
- L'analyse sémantique du code, qui consiste à identifier à quel niveau (locale, globale) nos variables existent. Le compilateur stocke ces informations, puis commence à créer une représentation intermédiaire du code.

Le *middle-end*, qui intervient ensuite, ne dépend de rien si ce n'est de la représentation intermédiaire choisie. Dans cette phase, les compilateurs vont manipuler cette représentation afin d'optimiser notre code et de le rendre plus performant.

Enfin, le *back-end*, lui, dépend de la machine cible mais pas du langage. En fonction de l'architecture de l'ordinateur qui doit exécuter les instructions, ou bien de s'il s'agit d'une machine virtuelle, les compilateurs doivent s'adapter. Pour une machine physique, ils produiront du code assembleur adapté à leur architecture. Pour une machine virtuelle, ils produiront du *bytecode*. [2]

Lors de la compilation d'une fonction, nous choisissons où stocker nos variables locales. Ce choix se fait au cours du *back-end*, puisqu'il dépend du stockage de la machine cible. On pourrait décider de tout ranger directement dans la pile, qui se situe dans la RAM [3], mais cela ne maximiserait pas nos performances. En effet, plus proche du processeur nous trouvons les **registres**, espaces de stockage de l'ordre de quelques kilo octet environ 300 fois plus rapide que la mémoire vive. C'est dans les registres que le processeur exécute ces opérations, et c'est à cet endroit qu'il est donc le plus judicieux de stocker nos variables. [4]

Cependant, il n'y a pas toujours assez de registres disponibles pour toutes les variables à stocker, ce qui nous amène au problème d'allocation de registre : comment attribuer les registres libres le plus efficacement possible ?

L'approche classique de ce problème est la **k-coloration** de graphe. En représentant chaque variable par un sommet du graphe et en reliant deux sommets si et seulement si la durée de vie des deux variables dans le programme se superpose, on obtient un graphe dont le but est d'associer à chaque nœud un registre différent de ces voisins. En représentant les registres par des couleurs, nous nous retrouvons bien dans un problème de k-coloration de graphe, avec k le nombre de registres disponibles. Si k est trop petit, il est alors possible de diviser une variable

non coloriable en deux, en séparant sa ligne de vie. Cela permet de réduire son nombre de voisin, et aide à l'attribution d'une couleur. Dans l'ordinateur, cela revient à la déplacer de registre, voir à la conserver dans la RAM. [5]

Le problème de k-coloration est un problème **NP-complet**, ce qui signifie qu'il peut dans certains cas être très coûteux à résoudre, mais aussi qu'il est un problème NP et donc qu'il peut se réduire à un problème SAT. De nos jours, de bons **SAT-solvers** existent basés sur l'algorithme DPLL. Des méthodes spécifiques à la k-coloration de graphe pour l'allocation de registre existent également, comme la méthode de Chaitin et al. [5]

Problématique retenue

Comment l'allocation de registre utilise la coloration des graphes d'interférences, et de quelle façon peut-on utiliser les graphes cordeaux ?

Objectifs du TIPE du candidat

1/ Modéliser des graphes de flot de contrôle pour différents programmes.

2/ Réaliser un sat-solveur pour la k-coloration de graphe, et comparer ses performances avec le programme minisat.

3/ Ajoutant la possibilité de mettre un nœud de côté pour le stocker dans une mémoire plus lente si la k-coloration n'est pas possible.

Références bibliographiques (ÉTAPE 1)

[1] ANDREW W.APPEL : Modern Compiler Implementation in C : - *Part I Fundamentals of Compilation - 10 Liveness Analysis - 11 Register Allocation*

[2] ROBERT NYSTROM : Crafting Interpreters : <https://craftinginterpreters.com/a-map-of-the-territory.html>

[3] : Pile (informatique) : [https://fr.wikipedia.org/wiki/Pile_\(informatique\)](https://fr.wikipedia.org/wiki/Pile_(informatique))

[4] : Hiérarchie de mémoire : https://fr.wikipedia.org/wiki/Hi%C3%A9rarchie_de_m%C3%A9moire

[5] PRESTON BRIGGS, KEITH D. COOPER, ET LINDA TORCZON : Improvements to Graph Coloring Register Allocation : <https://dl.acm.org/doi/10.1145/177492.177575>

DOT

[1] : En août, recherche sur la compilation

[2] : En octobre, découverte du problème d'allocation de registre et début des recherches

[3] : *En décembre, programmation d'un SAT-solver*

[4] : *En janvier, implémentation d'une modification du graphe d'interférence*

[5] : *En mars, approfondissement de ma compréhension du sujet*